

Contrato Mínimo API — Jogo de Cartas Distribuído UNO

CONTRATO MÍNIMO V1.1 - 13-06-2026

Formato das mensagens: JSON

Comunicação sugerida: HTTP REST

1. Observação sobre códigos de terminal

Os códigos curtos podem ser usados na interface de terminal, por exemplo:

R3 = carta vermelha número 3

B5 = carta azul número 5

Porém, esses códigos não são o formato oficial.

Na comunicação entre cliente e servidor, todas as cartas devem ser representadas em JSON.

Exemplo oficial:

```
{  
  "id": "carta-001",  
  "cor": "VERMELHO",  
  "tipo": "NUMERICA",  
  "valor": "3"  
}
```

2. Vocabulário padronizado

Cores

AMARELO

AZUL

VERMELHO

VERDE

PRETO

A cor PRETO será usada para cartas do tipo CORINGA e MAIS_QUATRO.

Quando uma carta CORINGA ou MAIS_QUATRO for jogada, a nova cor da rodada será informada pelo campo corEscolhida.

Tipos de carta

NUMERICA
PULAR
INVERTER
MAIS_DOIS
CORINGA
MAIS_QUATRO

Status da partida

AGUARDANDO_JOGADORES
EM_ANDAMENTO
FINALIZADO
CANCELADO

Sentido da rodada

HORARIO
ANTI_HORARIO

Tipos de evento

JOGADOR_ENTROU
JOGO_INICIADO
CARTA_JOGADA
CARTA_COMPRADA
UNO_CHAMADO
PENALIDADE_UNO
JOGADOR_BATEU
JOGO_FINALIZADO
FAILOVER
LIDER_ALTERADO

3. Modelo oficial de carta

Carta numérica

```
{  
  "id": "carta-001",  
  "cor": "VERMELHO",  
  "tipo": "NUMERICA",  
  "valor": "3"  
}
```

Carta Pular

```
{  
  "id": "carta-002",  
  "cor": "AZUL",  
  "tipo": "PULAR",  
  "valor": null  
}
```

Carta Inverter

```
{  
  "id": "carta-003",  
  "cor": "VERDE",  
  "tipo": "INVERTER",  
  "valor": null  
}
```

Carta +2

```
{  
  "id": "carta-004",  
  "cor": "AMARELO",  
  "tipo": "MAIS_DOIS",  
  "valor": null  
}
```

Carta Coringa

```
{  
  "id": "carta-005",  
  "cor": "PRETO",  
  "tipo": "CORINGA",  
  "valor": null  
}
```

Carta +4

```
{  
  "id": "carta-006",  
  "cor": "PRETO",  
  "tipo": "MAIS_QUATRO",  
  "valor": null  
}
```

4. Resposta padrão de sucesso

```
{  
  "sucesso": true,  
  "mensagem": "Operação realizada com sucesso",  
  "dados": {}  
}
```

5. Resposta padrão de erro

```
{  
  "sucesso": false,  
  "erro": {  
    "codigo": "JOGADA_INVALIDA",  
    "mensagem": "A carta jogada não possui a mesma cor, valor ou símbolo da carta do  
topo."  
  }  
}
```

6. Códigos de erro padronizados

JOGO_NAO_ENCONTRADO
JOGADOR_NAO_ENCONTRADO
JOGO_CHEIO
JOGO_JA_INICIADO
NAO_E_SUA_VEZ
CARTA_NAO_ENCONTRADA
JOGADA_INVALIDA
COR_OBRIGATORIA
JOGADOR_NAO_ESTA_COM_UMA_CARTA
JOGADOR_AINDA_TEM_CARTAS
JOGO_FINALIZADO
SERVIDOR_NAO_E_LIDER
NOME_INVALIDO
ERRO_INTERNO

7. Endpoints mínimos

7.1 GET /servidor

Retorna informações sobre o servidor.

Cliente envia

Nada.

Servidor responde

```
{
  "sucesso": true,
  "dados": {
    "servidorId": "srv-01",
    "nome": "Servidor Grupo 1",
    "versaoContrato": "1.0",
    "status": "ATIVO",
    "lider": true,
    "enderecoLider": "http://localhost:8080",
    "versaoEstadoAtual": 15
  }
}
```

Erros possíveis

SERVIDOR_NAO_E_LIDER
ERRO_INTERNO

Observação: servidor indisponível, timeout ou falha de conexão não retornam necessariamente JSON. São erros tratados pelo cliente.

7.2 POST /jogadores

Cria um jogador.

Cliente envia

```
{
  "nome": "Ana"
}
```

Servidor responde

```
{
  "sucesso": true,
  "mensagem": "Jogador criado com sucesso",
  "dados": {
    "jogadorId": "jogador-001",
    "nome": "Ana"
  }
}
```

Erros possíveis

NOME_INVALIDO
ERRO_INTERNO

7.3 POST /jogos

Cria uma nova partida.

Cliente envia

```
{  
  "jogadorId": "jogador-001"  
}
```

Servidor responde

```
{  
  "sucesso": true,  
  "mensagem": "Jogo criado com sucesso",  
  "dados": {  
    "gameId": "jogo-001",  
    "status": "AGUARDANDO_JOGADORES"  
  }  
}
```

Erros possíveis

JOGADOR_NAO_ENCONTRADO
ERRO_INTERNO

7.4 GET /jogos

Lista os jogos existentes.

Cliente envia

Nada.

Servidor responde

```
{
  "sucesso": true,
  "dados": [
    {
      "gameId": "jogo-001",
      "status": "AGUARDANDO_JOGADORES",
      "quantidadeJogadores": 2,
      "maxJogadores": 4
    }
  ]
}
```

Erros possíveis

ERRO_INTERNO

7.5 POST /jogos/{gameId}/entrar

Permite que um jogador entre em uma partida.

Cliente envia

```
{
  "jogadorId": "jogador-002"
}
```

Servidor responde

```
{
  "sucesso": true,
  "mensagem": "Jogador entrou na partida",
  "dados": {
    "gameId": "jogo-001",
    "status": "AGUARDANDO_JOGADORES",
    "quantidadeJogadores": 2
  }
}
```

Erros possíveis

JOGO_NAO_ENCONTRADO
JOGADOR_NAO_ENCONTRADO
JOGO_CHEIO
JOGO_JA_INICIADO

7.6 GET /jogos/{gameId}/estado

Consulta o estado visível da partida.

Como este endpoint é GET, o jogadorId deve ser enviado como parâmetro na URL.

Exemplo:

GET /jogos/jogo-001/estado?jogadorId=jogador-001


```
Servidor responde {
  "sucesso": true,
  "dados": {
    "gameId": "jogo-001",
    "status": "EM_ANDAMENTO",
    "versaoEstado": 10,
    "jogadorDaVez": "jogador-002",
    "sentido": "HORARIO",
    "corAtual": "AZUL",
    "cartaTopo": {
      "id": "carta-010",
      "cor": "AZUL",
      "tipo": "NUMERICA",
      "valor": "5"
    },
    "minhaMao": [
      {
        "id": "carta-011",
        "cor": "AZUL",
        "tipo": "PULAR",
        "valor": null
      }
    ],
    "jogadores": [
      {
        "jogadorId": "jogador-001",
        "nome": "Ana",
        "quantidadeCartas": 1,
        "chamouUno": true
      },
      {
        "jogadorId": "jogador-002",
        "nome": "Bruno",
        "quantidadeCartas": 4,
        "chamouUno": false
      }
    ]
  }
}
```

Erros possíveis

JOGO_NAO_ENCONTRADO
JOGADOR_NAO_ENCONTRADO

7.7 POST /jogos/{gameId}/jogarCarta

Realiza uma jogada.

Cliente envia

```
{
  "jogadorId": "jogador-001",
  "cartaId": "carta-011",
  "corEscolhida": null
}
```

Para CORINGA ou MAIS_QUATRO, o campo corEscolhida é obrigatório:

```
{
  "jogadorId": "jogador-001",
  "cartaId": "carta-050",
  "corEscolhida": "VERDE"
}
```

Servidor responde

```
{
  "sucesso": true,
  "mensagem": "Carta jogada com sucesso",
  "dados": {
    "gameId": "jogo-001",
    "versaoEstado": 11,
    "proximoJogador": "jogador-002",
    "corAtual": "VERDE"
  }
}
```

Erros possíveis

JOGO_NAO_ENCONTRADO
JOGADOR_NAO_ENCONTRADO
NAO_E_SUA_VEZ
CARTA_NAO_ENCONTRADA
JOGADA_INVALIDA
COR_OBRIGATORIA
JOGO_FINALIZADO

7.8 POST /jogos/{gameId}/comprar

Permite ao jogador comprar uma carta.

Regra definida

Ao comprar uma carta, o jogador perde a vez.
Ele não poderá jogar imediatamente a carta comprada.

Cliente envia

```
{  
  "jogadorId": "jogador-001"  
}
```

Servidor responde

```
{  
  "sucesso": true,  
  "mensagem": "Carta comprada com sucesso",  
  "dados": {  
    "cartaComprada": {  
      "id": "carta-020",  
      "cor": "AMARELO",  
      "tipo": "NUMERICA",  
      "valor": "8"  
    },  
    "passouAVez": true,  
    "proximoJogador": "jogador-003"  
  }  
}
```

Erros possíveis

JOGO_NAO_ENCONTRADO
JOGADOR_NAO_ENCONTRADO
NAO_E_SUA_VEZ
JOGO_FINALIZADO

7.9 POST /jogos/{gameId}/uno

Registra que o jogador chamou UNO.

Cliente envia

```
{  
  "jogadorId": "jogador-001"  
}
```

Servidor responde

```
{
  "sucesso": true,
  "mensagem": "UNO chamado com sucesso",
  "dados": {
    "jogadorId": "jogador-001",
    "quantidadeCartas": 1
  }
}
```

Erros possíveis

JOGO_NAO_ENCONTRADO
JOGADOR_NAO_ENCONTRADO
JOGADOR_NAO_ESTA_COM_UMA_CARTA
JOGO_FINALIZADO

7.10 POST /jogos/{gameId}/bater

Confirma a vitória do jogador.

Regra definida

Bater ocorre quando o jogador fica com zero cartas após uma jogada válida.

Cliente envia

```
{
  "jogadorId": "jogador-001"
}
```

Servidor responde

```
{
  "sucesso": true,
  "mensagem": "Jogador bateu",
  "dados": {
    "vencedor": "jogador-001",
    "status": "FINALIZADO"
  }
}
```

Erros possíveis

JOGO_NAO_ENCONTRADO
JOGADOR_NAO_ENCONTRADO
JOGADOR_AINDA_TEM_CARTAS
JOGO_FINALIZADO

7.11 GET /jogos/{gameId}/eventos

Retorna os eventos da partida.

Como este endpoint é GET, o campo desde deve ser enviado como parâmetro na URL.

Exemplo:

GET /jogos/jogo-001/eventos?desde=0

Servidor responde

```
{
  "sucesso": true,
  "dados": [
    {
      "sequencia": 1,
      "tipo": "JOGADOR_ENTROU",
      "jogadorId": "jogador-001",
      "mensagem": "Ana entrou na partida",
      "versaoEstado": 1
    },
    {
      "sequencia": 2,
      "tipo": "CARTA_JOGADA",
      "jogadorId": "jogador-001",
      "mensagem": "Ana jogou uma carta",
      "versaoEstado": 2
    }
  ]
}
```

Erros possíveis

JOGO_NAO_ENCONTRADO
ERRO_INTERNO

7.12 GET /leaderboard

Retorna o ranking de vitórias.

Cliente envia

Nada.

Servidor responde

```
{
  "sucesso": true,
  "dados": [
    {
      "jogadorId": "jogador-001",
      "nome": "Ana",
      "vitorias": 3
    },
    {
      "jogadorId": "jogador-002",
      "nome": "Bruno",
      "vitorias": 1
    }
  ]
}
```

Erros possíveis

ERRO_INTERNO

8. Regras mínimas

O servidor é responsável por validar todas as jogadas.

O cliente não deve decidir sozinho se uma carta pode ser jogada.

O cliente deve enviar apenas jogadorId, cartald e, se necessário, corEscolhida.

A mão dos outros jogadores não deve ser enviada ao cliente.

O cliente só vê a própria mão.

Os demais jogadores aparecem apenas com quantidadeCartas.

Comprar carta passa a vez.

A carta comprada não pode ser jogada imediatamente.

CORINGA pode ser jogado a qualquer momento.

MAIS_QUATRO pode ser jogado a qualquer momento.

CORINGA e MAIS_QUATRO exigem corEscolhida.

MAIS_DOIS faz o próximo jogador comprar duas cartas e perder a vez.

MAIS_QUATRO faz o próximo jogador comprar quatro cartas e perder a vez.

PULAR faz o próximo jogador perder a vez.

INVERTER muda o sentido da rodada.

Em partida com dois jogadores, INVERTER funciona como PULAR.

UNO deve ser chamado quando o jogador ficar com uma carta.

Se o jogador não chamar UNO, recebe penalidade de duas cartas.

Bater ocorre quando o jogador fica com zero cartas após uma jogada válida.

O estado da partida deve possuir versaoEstado.

Os eventos devem possuir sequencia.

O servidor deve informar se é líder no endpoint GET /servidor.

9. Informações mínimas do estado da partida

O servidor deve manter internamente:

```
{
  "gameId": "jogo-001",
  "status": "EM_ANDAMENTO",
  "versaoEstado": 10,
  "jogadores": [],
  "jogadorDaVez": "jogador-002",
  "sentido": "HORARIO",
  "corAtual": "AZUL",
  "cartaTopo": {},
  "monteCompra": [],
  "monteDescarte": [],
  "eventos": [],
  "vencedor": null
}
```

Nem todo esse estado deve ser exposto ao cliente.

O cliente deve receber apenas as informações públicas da partida e a própria mão.

10. Regras para integração entre grupos

Cada grupo deverá documentar:

Endereço do servidor
Porta utilizada
Como iniciar o servidor
Como iniciar o cliente
Como criar jogador
Como criar partida
Como entrar em partida
Como jogar carta
Como comprar carta
Como chamar UNO
Como consultar eventos
Como consultar leaderboard
Como testar failover

CONTRATO MÍNIMO V1 - 11-06-2026

Quais informações são necessárias para uma jogada acontecer?

- ID DO JOGADOR
- QUAL COR DA CARTA
- QUAL NUMERAL DA CARTA
- SE É UMA CARTA ESPECIAL
- QUAL CARTA ESTÁ NO TOPO DO MONTE
- QUANTIDADE DE CARTA DE CADA JOGADOR
- NÚMERO DA RODADA
- ID DA SESSÃO/JOGO
- NÚMERO TOTAL DE JOGADORES DA SESSÃO
- ARRAY DE JOGADORES DA SESSÃO PARA ORDEM DO JOGO
- INFORMAÇÃO SE A JOGADA É UNO
- SE FOI CHAMADO UNO
- CONTADOR DE CARTAS EM JOGO E NO MONTE
-

Definição de vocabulário comum:

CORES, TIPO DE CARTA, STATUS DE PARTIDA, TIPO DE EVENTO

Cor	ID (primeiro dígito)
-----	----------------------

Amarelo	Y
Azul	B
Verde	G
Vermelho	R

Tipo de Carta	ID (segundo dígito)	
Numerais (0 a 9)	0 - 9	Passar primeiro o id da cor desejada e após a carta esperada.
Reverso	R	
Coringa	X	Se o jogador usar um coringa por ex, deve se passar a cor escolhida no 1 dígito e em seguida o id da carta ex: GX = coringa verde
+4	Y	
+2	Z	
Bloqueio	B	

carta	1 dígito	2 dígito	final
3 vermelho	R - red/vermelho	3	r3
reverso verde	G - verde/green	R	gr
coringa (passando amarelo)	Y- yellow/amarelo	X	YX
+4 passando azul	B - Blue/Azul	Y	BY

Tipo de Carta	ID (segundo dígito)		
Numerais (0 a 9)	0 - 9		
CORINGA	TIPO	COR	FINAL
J = Joker/Coringa	R = Reverse	(R,B,G,Y)	JR(Cor selecionada)
J = Joker/Coringa	B = Block/Pulo	(R,B,G,Y)	JB(Cor selecionada)
J = Joker/Coringa	+2	(R,B,G,Y)	J+2(Cor Selecionada)
J = Joker/Coringa	+4	(R,B,G,Y)	J+4(Cor Selecionada)

J = Joker/Coringa	T = Trade	(R,B,G,Y)	JT(Cor selecionada)
-------------------	-----------	-----------	---------------------

Formatos padrões (JSON) De Envios?

Formatos padrões (JSON) de Respostas?

```
{
  "sucesso": true,
  "mensagem": "Operação realizada com sucesso",
  "dados": {}
}
```

Para cada endpoint precisamos responder:

- O que o cliente deve enviar?
- O que o servidor deve responder?
- Quais erros podem acontecer?
 - Tratar comandos incorretos
 - Tratar erros de conexão e timeout

POST /jogadores (qual jogo eu quero saber quem está jogando / quais jogadores estão jogando / partida não encontrada)

POST /jogos

GET /jogos

POST /jogos/{gameId}/entrar (preciso passar a id do jogo/ deve informar se entrei no jogo ou não)

GET /jogos/{gameId}/estado

POST /jogos/{gameId}/jogarCarta (qual carta, qual usuários jogou / pode não ser minha vez, a carta não foi aceita, carta errada)

POST /jogos/{gameId}/comprar (usuário que comprou / qual carta voltou)

POST /jogos/{gameId}/uno (usuário que está uno / validação se ele realmente está uno)

POST /jogos/{gameId}/bater

GET /jogos/{gameId}/eventos (

GET /leaderboard (retornar usuário e partidas ganhas)

GET /servidor

Usuario {

id: string;

nome: string;

vitorias: int;

}

```

Hand: {
    Cartas: Array[Carta];
    id_Usuario: Usuario;
    id_Partida: Partida;
}

Carta {
    cor: ENUM (null | azul | vermelho | verde | amarelo);
    numero: ( null | 0...9)
    efeito: ENUM (null | adicao | reverso | troca_de_cor);
    efeito_valor: {
        cor_alvo: ENUM ( null | azul | vermelho | verde | amarelo )
        valor_adicao: ENUM (null | +2 | +4)
    }
}

Baralho {
    id: string;
    cartas: Array [Carta];
}

Partida {
    id: string;
    status: ENUM ("em andamento" | "prestes a começar" | "em espera" |
"finalizada");
    jogadores: Array [Usuario];
    rodada_atual: -Contador++ (se alinhará com posição de array de usuários)
    vencedor: Usuario;
    pilha_de_cartas: Array [Carta];
}

```

INFORMAÇÕES PARA VOCÊS PENSAREM E BATEREM MARTELO:

Endpoint	O que o cliente envia?	O que o servidor responde?	Quais erros podem ocorrer?
GET /servidor	Nada	Dados do servidor	Servidor indisponível
POST /jogadores	Nome	ID do jogador	Nome inválido

POST /jogos	jogadorId	gameId	Jogador não encontrado
GET /jogos	Nada	Lista de jogos	Erro interno
POST /jogos/{gameId}/entrar	jogadorId	Confirmação	Jogo cheio
GET /jogos/{gameId}/estado	jogadorId	Estado visível	Jogo não encontrado
POST /jogos/{gameId}/jogarCarta	jogadorId, cartId, corEscolhida	Jogada aceita	Não é sua vez
POST /jogos/{gameId}/comprar	jogadorId	Carta comprada	Não é sua vez
POST /jogos/{gameId}/uno	jogadorId	UNO registrado	Jogador não tem uma carta
POST /jogos/{gameId}/bater	jogadorId	Vitória confirmada	Jogador ainda tem cartas
GET /jogos/{gameId}/eventos	desde	Lista de eventos	Jogo não encontrado
GET /leaderboard	Nada	Ranking	Erro interno

ERROS

```
{
  "sucesso": false,
  "erro": {
    "codigo": "JOGADA_INVALIDA",
    "mensagem": "A carta jogada não possui a mesma cor, valor ou símbolo da carta do topo."
  }
}
```

CÓDIGO DE ERROS:

```
JOGO_NAO_ENCONTRADO
JOGADOR_NAO_ENCONTRADO
JOGO_CHEIO
JOGO_JA_INICIADO
```

NAO_E_SUA_VEZ
CARTA_NAO_ENCONTRADA
JOGADA_INVALIDA
COR_OBRIGATORIA
JOGADOR_NAO_ESTA_COM_UMA_CARTA
JOGO_FINALIZADO

RESPOSTA PADRÃO

```
{  
  "sucesso": true,  
  "mensagem": "Operação realizada com sucesso",  
  "dados": {}  
}
```

```
get /servidor  
{  
  "sucesso": true,  
  "dados": {  
    "servidorId": "srv-01",  
    "nome": "Servidor Grupo 1",  
    "versaoContrato": "1.0",  
    "status": "ATIVO",  
    "lider": true,  
    "enderecoLider": "http://localhost:8080",  
    "versaoEstadoAtual": 15  
  }  
}
```

```
get /leaderboard  
{  
  "sucesso": true,  
  "dados": [  
    {  
      "jogadorId": "jogador-001",  
      "nome": "Ana",  
      "vitorias": 3  
    },  
    {  
      "jogadorId": "jogador-002",  
      "nome": "Bruno",  
      "vitorias": 1  
    }  
  ]  
}
```

```
}
```

Para replicação:

GET /jogos/{gameId}/eventos

```
{
  "sucesso": true,
  "dados": [
    {
      "sequencia": 1,
      "tipo": "JOGADOR_ENTROU",
      "jogadorId": "jogador-001",
      "mensagem": "Ana entrou na partida",
      "versaoEstado": 1
    },
    {
      "sequencia": 2,
      "tipo": "CARTA_JOGADA",
      "jogadorId": "jogador-001",
      "mensagem": "Ana jogou uma carta",
      "versaoEstado": 2
    }
  ]
}
```

POST /jogos/{gameId}/bater

```
{
  "jogadorId": "jogador-001"
}
```

resposta:

```
{
  "sucesso": true,
  "mensagem": "Jogador bateu",
  "dados": {
    "vencedor": "jogador-001",
    "status": "FINALIZADO"
  }
}
```

erros possíveis:

JOGADOR_AINDA_TEM_CARTAS
JOGO_NAO_ENCONTRADO

JOGO_FINALIZADO

post /jogos/{gameId}/uno

```
{
  "jogadorId": "jogador-001"
}
```

resposta:

```
{
  "sucesso": true,
  "mensagem": "UNO chamado com sucesso",
  "dados": {
    "jogadorId": "jogador-001",
    "quantidadeCartas": 1
  }
}
```

post /jogos/{gameId}/comprar

```
{
  "jogadorId": "jogador-001"
}
```

resposta:

```
{
  "sucesso": true,
  "mensagem": "Carta comprada com sucesso",
  "dados": {
    "cartaComprada": {
      "id": "carta-020",
      "cor": "AMARELO",
      "tipo": "NUMERICA",
      "valor": "8"
    },
    "passouAVez": true,
    "proximoJogador": "jogador-003"
  }
}
```

importante estabelecer que ao comprar passa a vez, não irá jogar a carta que pescou

POST /jogos/{gameId}/jogarCarta

envia

```
{
```

```
"jogadorId": "jogador-001",
"cartaId": "cartax",
"corEscolhida": null
} (precisam definir como será o padrão das cartas...)
```

SUGESTÃO:

```
{
  "id": "carta-001",
  "cor": "VERMELHO",
  "tipo": "NUMERICA",
  "valor": "3"
}
resposta
{
  "sucesso": true,
  "mensagem": "Carta jogada com sucesso",
  "dados": {
    "gameId": "jogo-001",
    "versaoEstado": 11,
    "proximoJogador": "jogador-002",
    "corAtual": "VERDE"
  }
}
```

erros possíveis:

JOGO_NAO_ENCONTRADO
JOGADOR_NAO_ENCONTRADO
NAO_E_SUA_VEZ
CARTA_NAO_ENCONTRADA
JOGADA_INVALIDA
COR_OBRIGATORIA
JOGO_FINALIZADO

GET /jogos/{gameId}/estado

```
{
  "sucesso": true,
  "dados": {
    "gameId": "jogo-001",
    "status": "EM_ANDAMENTO",
    "versaoEstado": 10,
    "jogadorDaVez": "jogador-002",
    "sentido": "HORARIO",
    "corAtual": "AZUL",
```



```
"cartaTopo": {
  "id": "carta-010",
  "cor": "AZUL",
  "tipo": "NUMERICA",
  "valor": "5"
},
"minhaMao": [
  {
    "id": "carta-011",
    "cor": "AZUL",
    "tipo": "PULAR",
    "valor": null
  }
],
"jogadores": [
  {
    "jogadorId": "jogador-001",
    "nome": "Ana",
    "quantidadeCartas": 1,
    "chamouUno": true
  },
  {
    "jogadorId": "jogador-002",
    "nome": "Bruno",
    "quantidadeCartas": 4,
    "chamouUno": false
  }
]
}
```

ERROS

JOGO_NAO_ENCONTRADO
JOGADOR_NAO_ENCONTRADO

POST /jogos/{gameId}/entrar

```
{
  "jogadorId": "jogador-002"
}
```

RESPOSTA

```
{
  "sucesso": true,
  "mensagem": "Jogador entrou na partida",
}
```

```
"dados": {  
  "gameId": "jogo-001",  
  "status": "AGUARDANDO_JOGADORES",  
  "quantidadeJogadores": 2  
}
```

os códigos podem ser usados para o terminal mas precisamos padronizar no json
AMARELO, AZUL, VERMELHO, VERDE E PRETO (CORINGAS)

UNO	PADRONIZAR
Número de 0 a 9	NUMERICA
Bloqueio / Pulo	PULAR
Reverso	INVERTER
+2	MAIS_DOIS
+4	MAIS_QUATRO

ESTADOS DA PARTIDA

AGUARDANDO_JOGADORES
EM_ANDAMENTO
FINALIZADO
CANCELADO

SENTIDO: HORARIO - ANTI_HORARIO

EVENTOS:
JOGADOR_ENTROU
JOGO_INICIADO
CARTA_JOGADA
CARTA_COMPRADA
UNO_CHAMADO
PENALIDADE_UNO
JOGADOR_BATEU
JOGO_FINALIZADO
FAILOVER
LIDER_ALTERADO